SCS 1101 · INTRODUCTION TO COMPUTER SCIENCE AND PROGRAMMING

The Calculator Program in C using switch-case

Two complete programs, a one-line calculator and a menu-driven calculator with a loop, explained the Group 13 way: like you're 3, model answer, algorithm and code, colour flowchart, real compiled output, engineering lens and examiner's tips.

SOLUTION NOTE · SWITCH-CASE

COURSE
SCS 1101 Introduction to Computer Science and Programming

TOPIC
Module 10 · Selection: if/else and switch

COMPILER
gcc, flags -Wall -Wextra: 0 warnings, all outputs below are real

PREPARED FOR
Group 13 · NUST Civil & Water Engineering

INSIDE

1. Like you're 3: what switch does
2. The switch statement: syntax and rules
3. Program 1: one-line calculator
4. Flowchart for Program 1
5. Real output and dry run
6. Program 2: menu-driven calculator
7. Flowchart and output for Program 2
8. switch vs if-else ladder
9. Common errors (with real bug output)
10. Engineering lens
11. Exam questions and examiner's tips

Both programs were compiled with gcc and run on the test inputs shown. The source files can be typed straight into Code::Blocks, Dev-C++ or any C compiler.

Group 13
Study Board

1 Like you're 3: what switch does

Imagine a lift in a building. You press a button: **1**, **2**, **3** or **G**. The lift looks at which button you pressed and goes straight to that floor. It does not stop at every floor asking "is it this one?". If you press a button that does not exist, it does nothing (that is the **default**).

A `switch` statement is that lift. It looks at **one value** (the operator `+`, `-`, `*`, `/` the user typed) and **jumps** to the matching `case`. The `break` is the lift doors opening: you get out and stop. Forget the `break` and the lift carries on to the next floors too.

> **KEY IDEA**
>
> Use `switch` when one variable is compared with **many fixed values** (menu choices, operators, grades by letter). Use `if-else` when you test **ranges or conditions** (`mark >= 75`).

2 The switch statement: syntax and rules

```
switch (expression)
{
    case constant1:
        statements;
        break;
    case constant2:
        statements;
        break;
    ...
    default:
        statements;
}
```

How it runs

1. The **expression** is evaluated once.
2. Its value is compared with each **case** label.
3. Control **jumps** to the matching case and runs its statements.
4. `break` exits the switch.
5. If nothing matches, `default` runs (if present).

#	Rule	Example
1	The expression must be an **integer type**: `int`, `char`, `short`, `long` or `enum`. Not `float`, `double` or a string.	`switch (op)` with `char op` ✔ `switch (x)` with `double x` ✘
2	Case labels must be **constant** integer expressions, not variables.	`case '+':` ✔ `case 5:` ✔ `case n:` ✘
3	No two case labels may have the **same value**.	two `case 1:` labels → compile error
4	Without `break`, execution **falls through** into the next case.	See Section 9
5	Several cases can share one block (deliberate fall-through).	`case '*': case 'x':`
6	`default` is optional and may appear anywhere, but is usually last.	handles invalid input
7	Switches can be **nested**; `break` leaves only the innermost switch.	Program 2

3 Program 1: one-line calculator

> **QUESTION (TYPICAL EXAM WORDING)**
>
> Write a C program that reads two numbers and an arithmetic operator (+, −, *, /, %) entered as one expression, such as `12 * 4`, and uses a `switch` statement to display the result. The program must report division by zero and invalid operators. [15 marks]

ALGORITHM · H-D-I-P-O-R

Block	Step
Header	Include `stdio.h`; start `main()`.
Declare	`char op` for the operator; `double a, b` for the numbers.
Input	Prompt the user; read `a`, `op`, `b` with one `scanf("%lf %c %lf")`.
Process	`switch (op)`: case `+` add; `-` subtract; `*` or `x` multiply; `/` check b ≠ 0 then divide; `%` check b ≠ 0 then integer remainder; default: invalid operator.
Output	Print the expression and its result to 2 decimal places, or the error message.
Return	`return 0;` to tell the operating system the program ended normally.

CODE · CALC1.C

```c
/* calc1.c - simple calculator using switch-case (SCS 1101) */
#include <stdio.h>

int main(void)
{
    char op;            /* the operator the user types: + - * / %  */
    double a, b;        /* the two numbers                          */

    printf("Enter an expression (e.g. 12 * 4): ");
    scanf("%lf %c %lf", &a, &op, &b);

    switch (op)
    {
        case '+':
            printf("%.2f + %.2f = %.2f\n", a, b, a + b);
            break;
        case '-':
            printf("%.2f - %.2f = %.2f\n", a, b, a - b);
            break;
        case '*':
        case 'x':                       /* two cases share one action */
            printf("%.2f * %.2f = %.2f\n", a, b, a * b);
            break;
        case '/':
            if (b == 0)
                printf("Error: division by zero is not allowed.\n");
            else
                printf("%.2f / %.2f = %.2f\n", a, b, a / b);
            break;
        case '%':
            if ((int)b == 0)
                printf("Error: modulus by zero is not allowed.\n");
            else
                printf("%d %% %d = %d\n", (int)a, (int)b, (int)a % (int)b);
            break;
        default:
            printf("Error: '%c' is not a valid operator.\n", op);
    }
    return 0;
}
```

Notes. The space before `%c` in `"%lf %c %lf"` skips whitespace so the operator is read correctly. `%%` prints a literal percent sign. `%` works only on integers in C, so both numbers are cast with `(int)`. `case '*': case 'x':` lets users type either symbol.

4 Flowchart for Program 1

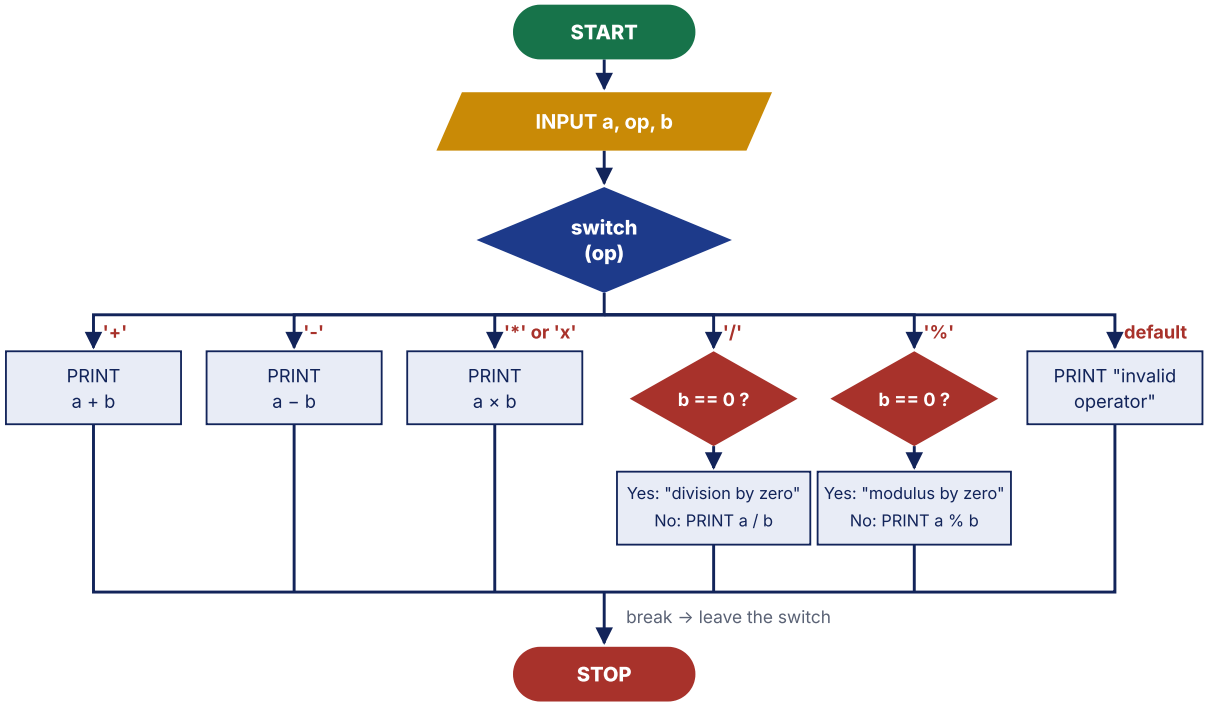


Figure 1. Flowchart for Program 1. Ovals = start/stop, parallelogram = input/output, diamonds = decisions, rectangles = processes. Every case path ends with break and joins the line to STOP.

5 Real output and dry run

Each line below is the actual output of `calc1` for the input shown (the prompt line is omitted).

Input typed	Case taken	Output printed
`12 * 4`	`'*'`	`12.00 * 4.00 = 48.00`
`15 / 4`	`'/'`, b ≠ 0	`15.00 / 4.00 = 3.75`
`9 / 0`	`'/'`, b = 0	`Error: division by zero is not allowed.`
`17 % 5`	`'%'`	`17 % 5 = 2`
`3 x 7`	`'x'` → falls into `'*'` block	`3.00 * 7.00 = 21.00`
`8 - 13.5`	`'-'`	`8.00 - 13.50 = -5.50`
`5 ? 2`	`default`	`Error: '?' is not a valid operator.`

DRY RUN (TRACE TABLE) FOR INPUT 15 / 4

Step	Statement	a	op	b	Output
1	`scanf(...)`	15.0	'/'	4.0	–
2	`switch (op)` → jumps to `case '/'`	15.0	'/'	4.0	–
3	`if (b == 0)` → false	15.0	'/'	4.0	–
4	`printf(... a / b)`	15.0	'/'	4.0	15.00 / 4.00 = 3.75
5	`break;` → `return 0;`	program ends			

6 Program 2: menu-driven calculator

QUESTION

Write a menu-driven calculator in C that repeatedly offers: 1 Add, 2 Subtract, 3 Multiply, 4 Divide, 5 Power, 6 Square root, 0 Exit. Use a `switch` statement for the menu and a loop so the user can do several calculations. Validate division by zero and square roots of negative numbers. [20 marks]

ALGORITHM · H-D-I-P-O-R

Block	Step
Header	Include `stdio.h` and `math.h` (for `pow`, `sqrt`); compile with `-lm` on Linux.
Declare	`int choice`; `double a, b, result`.
Input	Inside a `do-while` loop: display the menu, read `choice`.
Process	`switch (choice)`: cases 1–5 read two numbers then a **nested switch** computes the result (4 checks b ≠ 0); case 6 reads one number and checks a ≥ 0; case 0 says goodbye; default rejects the choice.
Output	Print the result to 4 decimal places or an error.
Return	Repeat while `choice != 0`; then `return 0;`.

CODE · CALC2.C

```c
/* calc2.c - menu-driven calculator: do-while loop + switch-case (SCS 1101) */
#include <stdio.h>
#include <math.h>

int main(void)
{
    int choice;
    double a, b, result;

    do
    {
        printf("\n===== CALCULATOR MENU =====\n");
        printf("1. Add          2. Subtract\n");
        printf("3. Multiply     4. Divide\n");
        printf("5. Power (a^b)  6. Square root\n");
        printf("0. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: case 2: case 3: case 4: case 5:
                printf("Enter two numbers: ");
                scanf("%lf %lf", &a, &b);
                switch (choice)                /* nested switch */
                {
                    case 1: result = a + b; break;
                    case 2: result = a - b; break;
                    case 3: result = a * b; break;
                    case 4:
                        if (b == 0) { printf("Error: cannot divide by zero.\n"); continue; }
                        result = a / b;
                        break;
                    case 5: result = pow(a, b); break;
                }
```

```
                printf("Result = %.4f\n", result);
                break;
            case 6:
                printf("Enter a number: ");
                scanf("%lf", &a);
                if (a < 0)
                    printf("Error: no real square root of a negative number.\n");
                else
                    printf("Result = %.4f\n", sqrt(a));
                break;
            case 0:
                printf("Goodbye.\n");
                break;
            default:
                printf("Invalid choice. Try 0 to 6.\n");
        }
    } while (choice != 0);

    return 0;
}
```

Notes. `case 1: case 2: case 3: case 4: case 5:` stacks five labels on one block so the two numbers are read once. Inside the nested switch, `continue` skips the `printf("Result…")` line after a division-by-zero error and goes straight to the loop test. A `do-while` is used because the menu must appear **at least once**.

7 Flowchart and output for Program 2

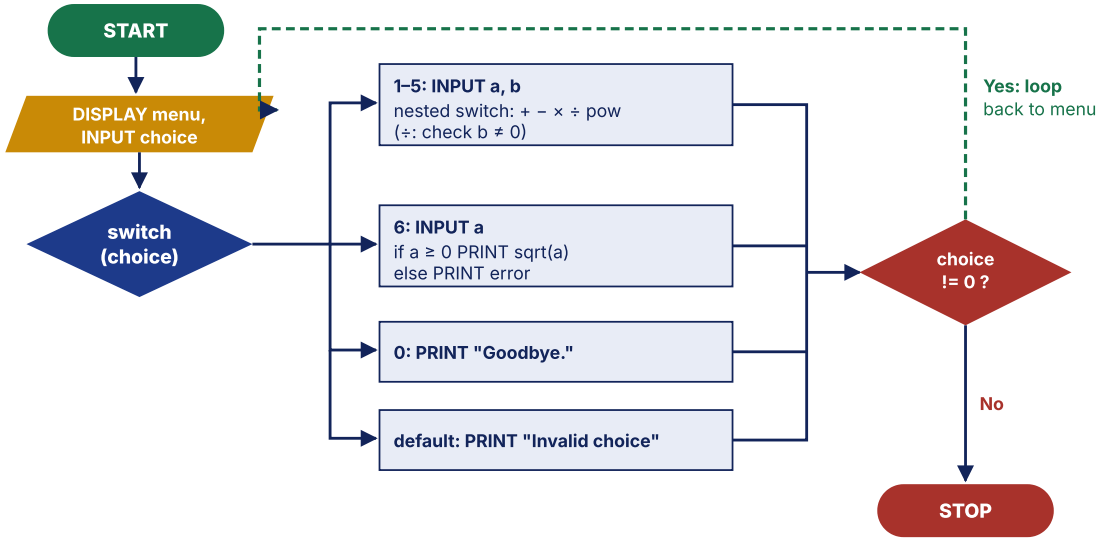


Figure 2. Flowchart for Program 2: a do-while loop around a switch. The loop test is at the bottom, so the menu always appears at least once.

Real session (inputs added in brackets; repeated menus shortened):

```
===== CALCULATOR MENU =====
1. Add          2. Subtract
3. Multiply     4. Divide
5. Power (a^b)  6. Square root
0. Exit
Enter your choice: [1]   Enter two numbers: [12 30]   Result = 42.0000
Enter your choice: [4]   Enter two numbers: [9 0]     Error: cannot divide by zero.
Enter your choice: [5]   Enter two numbers: [2 10]    Result = 1024.0000
Enter your choice: [6]   Enter a number: [144]        Result = 12.0000
Enter your choice: [6]   Enter a number: [-4]         Error: no real square root of a negative number.
Enter your choice: [7]   Invalid choice. Try 0 to 6.
Enter your choice: [0]   Goodbye.
```

8 switch vs if-else ladder

Point	switch	if-else-if ladder
Tests	Equality of one expression with constants	Any condition: ranges, &&, ||, floats
Data types	Integer types and char only	Any type
Readability	Very clear for menus and many fixed options	Long and repetitive for many options
Speed	Compiler can use a jump table: fast for many cases	Tests conditions one by one
Fall-through	Possible (useful or a bug)	Not possible
Best for	Calculators, menus, grade letters, days of week	Grade from a mark range, tax bands, limits

The same `'+'` and `'-'` logic as an if-else ladder:

```
if (op == '+')       printf("%.2f\n", a + b);
else if (op == '-')  printf("%.2f\n", a - b);
else                 printf("Invalid operator\n");
```

9 Common errors

ERROR 1 · FORGETTING BREAK (REAL OUTPUT)

With `op = '+'`, `a = 6`, `b = 2` and no `break` after the first two cases, the program printed all three results, because execution fell through:

```
case '+': printf("Sum = %.2f\n", a + b);        /* break forgotten */
case '-': printf("Difference = %.2f\n", a - b); /* break forgotten */
case '*': printf("Product = %.2f\n", a * b);
          break;
```

```
Sum = 8.00
Difference = 4.00
Product = 12.00
```

Error	Why it is wrong	Fix
`switch (x)` with `double x`	Compile error: switch quantity not an integer	Use int/char, or if-else
`case "+":`	Double quotes make a string, not a char	`case '+':`
`case n:` (variable)	Labels must be constants	Use a literal or `#define`
Two `case 1:`	Duplicate case value: compile error	Make labels unique
`scanf("%lf%c%lf")`	`%c` reads the space, not the operator	`"%lf %c %lf"`
No division-by-zero check	Gives `inf` or a crash for integers	`if (b == 0)` before dividing
`%` on doubles	`%` works only on integers	Cast with `(int)` or use `fmod()`
Missing `-lm`	Linker error: undefined reference to `pow` / `sqrt` on Linux	`gcc calc2.c -lm`

10 Engineering lens

Engineers use exactly this pattern in site tools and lab software: one menu, many fixed options. A civil engineering example is a **units and properties calculator**: `case 1` converts kN to tonnes, `case 2` computes the area of a circular pipe ($\pi d^2/4$), `case 3` computes flow Q = A × v, `case 4` computes concrete volume for a slab. Embedded controllers use switch for **state machines**: a traffic-light controller switches on the current state (RED, GREEN, AMBER); a pump controller switches on the tank level band (LOW, NORMAL, HIGH). The validation in this calculator (no division by zero, no square root of a negative) is the same habit as checking inputs before a structural calculation: bad input must be caught before it produces a confident wrong answer.

11 Exam questions and examiner's tips

PRACTICE QUESTIONS

1. Explain the purpose of the `break` and `default` statements in a switch. [4]
2. State three rules that apply to case labels in C. [3]
3. Write a C program using switch that reads two integers and an operator and prints the result. [15]
4. Draw a flowchart for the program in Q3. [6]
5. Compare the switch statement with the if-else-if ladder, giving one situation where each is better. [6]
6. What is printed by the "missing break" code in Section 9 when `op = '-'`? Explain. [4]
7. Write a menu-driven program using do-while and switch for converting units (m to ft, kg to lb, °C to °F). [15]

ANSWERS TO THE SHORT QUESTIONS

Q1. `break` ends the switch so execution continues after the closing brace; without it control falls through into the next case. `default` runs when no case matches, so it handles invalid input. **Q2.** Labels must be integer (or char) constants; no two labels may have the same value; they cannot be variables, ranges, floats or strings. **Q6.** It prints `Difference = 4.00` and `Product = 12.00`: control jumps to `case '-'`, and with no break it falls through into `case '*'` until the break. `Sum` is not printed because the jump skips the cases above.

EXAMINER'S TIPS

- Marks are usually split: headers and declarations (2), input (2), correct switch with breaks (5), validation (2), output format (2), comments and layout (2).
- Always include a `default` case; examiners look for it.
- Put `break` after **every** case unless you are deliberately stacking labels, and say so in a comment.
- Use single quotes for char cases: `'+'`.
- Check division by zero; it is the most common missing mark.
- Indent each case consistently and comment the purpose of the program at the top.
- If asked for a flowchart, use the correct symbols and show every case path joining before STOP.